

Secure Software Development – 2025/26

Code Analysis and Remediation with Static Application Security Testing

Ricardo Gonçalves

Carlos Oliveira | a20530

Nelson Araújo | a20743

João Cunha | a21598

June 2026

1. Introduction, Project and SAST Tools

This report covers the second part of the Secure Software Development assignment: detecting and remediating source code vulnerabilities with Static Application Security Testing (SAST). The target is InsecureWebApp (<https://github.com/kadraman/InsecureWebApp>), an intentionally vulnerable web application written in Python 3 with Flask, Jinja2 templates, an SQLite database accessed through raw SQL, and a Dockerfile. It was chosen because it concentrates many distinct vulnerability classes, from remote code execution to insecure configuration, in a small codebase that can be fully remediated within the scope of this work.

Two SAST tools were used: Bandit, a Python specific analyser, and Semgrep, a multi language rule based engine run with its automatic ruleset. Each tool was run twice, once on the original code to establish a baseline and once on the corrected version to measure the effect of the fixes. Using two independent tools improves coverage, and a finding reported by both is strong evidence of a genuine weakness, which is why such findings were prioritised.

2. Selected Vulnerabilities

Nine distinct vulnerabilities were selected and divided three per group member, prioritising the most critical classes; each item is supported by at least one scan and no vulnerability type is repeated. The first four were reported by both Bandit and Semgrep. The Member column identifies the group member responsible for each item.

#	Vulnerability	Bandit	Semgrep	Severity	Member
1	OS Command Injection	B605	dangerous-system-call	Critical	Carlos Oliveira
2	Insecure Deserialization (pickle)	B301 / B403	avoid-pickle	Critical	Carlos Oliveira
3	SQL Injection	B608	tainted-sql-string	High	Nelson Araújo
4	Arbitrary Code Execution (exec)	B102	exec-detected	Critical	João Cunha
5	Server-Side Template Injection	-	direct-use-of-jinja2	High	Nelson Araújo
6	Path Manipulation /	-	make-response-with-	High	João Cunha

#	Vulnerability	Bandit	Semgrep	Severity	Member
	unsafe file response		unknown-content		
7	Insecure Cookie Attributes	-	secure-set-cookie	Medium	Carlos Oliveira
8	Hardcoded Secret Key	B105	-	Medium	Nelson Araújo
9	Container Runs as Root	-	missing-user	Medium	João Cunha

3. Analysis and Remediation

Each vulnerability was corrected at the source level by removing the dangerous behaviour rather than filtering input. The before and after snippets below summarise each fix; the full rationale is in the accompanying supporting document.

1. OS Command Injection (Carlos Oliveira) [Bandit B605, Semgrep dangerous-system-call, Critical]

before:

```
arguments = request.args.get('arguments')
cmd = home.join(INITCMD).join(arguments)
os.system(cmd)
```

after:

```
action = request.args.get('action', 'status')
if action == 'status': output = os.getloadavg()
elif action == 'disk': output = shutil.disk_usage('/')
else: abort(400)
```

The arguments query parameter was concatenated into a string and passed to `os.system`, which runs through a shell. Shell metacharacters such as `;` or `|` would let an attacker append their own commands and execute them on the server, a full host compromise. The fix removes `os.system` entirely and exposes only a fixed allowlist of safe actions implemented with native Python functions, so there is no shell and no attacker controlled command string.

2. Insecure Deserialization (Carlos Oliveira) [Bandit B301/B403, Semgrep avoid-pickle, Critical]

before:

```
b64 = request.cookies.get('rememberme')
a = pickle.loads(base64.b64decode(b64))
session['email'] = a.email
```

after:

```
s = URLSafeSerializer(current_app.config['SECRET_KEY'], salt='rememberme')
try: data = s.loads(request.cookies.get('rememberme'))
except BadSignature: abort(400)
session['email'] = data['email']
```

The client controlled `rememberme` cookie was base64 decoded and passed to `pickle.loads`. Pickle can execute arbitrary code while reconstructing an object, so a crafted cookie alone leads to remote code

execution. The fix replaces pickle with a token signed using the application secret; the signature is verified before the data is used, so any tampered or forged cookie is rejected.

3. SQL Injection (Nelson Araújo) [Bandit B608, Semgrep tainted-sql-string, High]

before:

```
keywords = request.args.get('keywords', '')
query = "SELECT * FROM products WHERE name LIKE '%" + keywords + "%"
db.execute(query).fetchall()
```

after:

```
db.execute(
    'SELECT * FROM products WHERE name LIKE ?',
    (f'%{keywords}%')).fetchall()
```

The keywords search parameter was concatenated directly into a SQL LIKE clause, so input such as a quote followed by SQL could change the query structure and read or modify other data. The fix uses parameterized queries: the input is sent to the driver as a bound value and can never be interpreted as SQL. The same pattern was applied to every affected statement.

4. Arbitrary Code Execution (João Cunha) [Bandit B102, Semgrep exec-detected, Critical]

before:

```
def source(path):
    exec(open(path).read())
```

after:

```
def source(path):
    requested = (SAFE_FILES_DIR / path).resolve()
    ... # path validated, no exec
    return requested.read_text()
```

The load_file route read a user selected file and executed its contents with exec, which allows arbitrary Python execution and was confirmed exploitable in local testing. The fix removes exec and instead returns the file content as plain text, preserving the feature without the code execution risk. The same function also validates the path to prevent the user from accessing unauthorized files with root permissions, the Path Manipulation issue (CWE-22) covered in item 6.

5. Server-Side Template Injection (Nelson Araújo) [Semgrep direct-use-of-jinja2, High]

before:

```
rendered_output = Jinja2_Template(file_content).render()
# template: {{ rendered_output|safe }}
```

after:

```
render_template(..., rendered_output=value)
# template: {{ rendered_output }} (auto-escaped)
```

User content was compiled as a Jinja2 template and rendered, and the template output used the safe filter, so an attacker could inject template expressions that the engine evaluates, leading to data disclosure or code execution. The fix stops compiling input as a template and passes it as ordinary data, and removing the safe filter restores Jinja2 auto escaping.

6. Path Manipulation (João Cunha) [Semgrep make-response-with-unknown-content, High]

before:

```
filename = request.args.get('filename')
contents = source(filename) # any path
```

after:

```
r = (SAFE_FILES_DIR / name).resolve()
if not str(r).startswith(str(SAFE_FILES_DIR)):
    abort(400)
```

The filename parameter was used to build a file path with no restriction, so traversal sequences such as `../etc/passwd` could read files outside the intended directory. The fix confines access to a fixed safe base directory: the requested path is resolved and rejected if it falls outside that directory. A traversal test then returned HTTP 400.

7. Insecure Cookie Attributes (Carlos Oliveira) [Semgrep secure-set-cookie, Medium]

before:

```
response.set_cookie('sessionid', session_id)
```

after:

```
response.set_cookie('sessionid', session_id,
    secure=True, httponly=True, samesite='Strict')
```

Cookies were created without the Secure, HttpOnly and SameSite attributes, which exposes session values to interception over plain HTTP, to theft by client side scripts, and to cross site request forgery. The fix sets all three flags, so the cookie is only sent over HTTPS, is hidden from JavaScript, and is not attached to cross site requests.

8. Hardcoded Secret Key (Nelson Araújo) [Bandit B105, Medium]

before:

```
SESSION_SECRET = '<hardcoded value>' # in main.py
```

after:

```
SECRET_KEY = os.environ.get('SESSION_SECRET')
if not SECRET_KEY: raise RuntimeError('SESSION_SECRET is required')
```

The session secret was written directly in source code, so anyone with read access to the repository could forge sessions and signed tokens, which would also defeat the deserialization fix above. The fix loads the secret from an environment variable and makes the application refuse to start if it is missing, preventing an insecure default.

9. Container Runs as Root (João Cunha) [Semgrep missing-user, Medium]

before:

```
# Dockerfile has no USER instruction (runs as root)
```

after:

```
RUN useradd -m appuser && chown -R appuser /usr/src/iwa
USER appuser
```

The Dockerfile defined no USER, so the container ran as root; if the application is compromised, running as root makes container escape and host impact far easier. The fix applies least privilege by creating a dedicated unprivileged user that owns the application files and switching to it before the app runs.

4. Scan Results: Before and After

Tool	Before	After	Resolved
Bandit	23 (1 High, 5 Medium, 17 Low)	13 (0 High, 0 Medium, 13 Low)	All High and Medium
Semgrep	33	17	16

After remediation every High and Medium Bandit finding was eliminated. The remaining Bandit items are low severity assert statements inside the test suite and one false positive (see section 5). On the Semgrep side, the resolved rules include dangerous-system-call, exec-detected, avoid-pickle, tainted-sql-string, sqlalchemy-execute-raw-query, direct-use-of-jinja2, make-response-with-unknown-content and missing-user. The residual Semgrep findings are out of scope hardening items such as missing CSRF tokens, missing subresource integrity attributes on external CDN assets, and cookies in a separate assistant module.

5. False Positive

Bandit reports B105 (hardcoded password) at auth_routes.py:109 for the string “Incorrect password.”. This is a false positive: the value is a user facing error message shown when authentication fails, not a credential. The detection is triggered only by the word “password” in the literal. No change is required, which illustrates that SAST output must always be validated by a human analyst before any fix is applied.

6. Conclusion

Using Bandit and Semgrep together, we identified nine distinct vulnerabilities, including four critical issues confirmed by both tools, and remediated each one at the source code level. The second scan confirms the improvement, with all High and Medium Bandit findings removed and the Semgrep count roughly halved. The exercise shows how SAST accelerates the discovery of insecure patterns, while human review remains essential to confirm exploitability and to discard false positives such as the one documented above.

7. Vulnerability Classification and Dependency CVEs

The nine application-level findings are classified below by CWE, using the CWE identifiers reported by Bandit and Semgrep for each finding. These are not CVEs in their own right (the findings belong to a custom application, not a published library), but each maps to a well-known CWE and is illustrated with a real-world CVE of the same class. The pinned dependencies in requirements.txt carry their own assigned CVEs, listed separately.

7.1 Application Threats – CWE Classification

#	Vulnerability	Severity	CWE	Real-world CVE example
1	OS Command Injection	Critical	CWE-78	CVE-2023-3519 (Citrix RCE via command injection)
2	Insecure Deserialization	Critical	CWE-502	CVE-2019-20477 (PyYAML unsafe

#	Vulnerability	Severity	CWE	Real-world CVE example
	(pickle)			deserialization)
3	SQL Injection	High	CWE-89	CVE-2023-34362 (MOVEit SQL injection)
4	Arbitrary Code Execution (exec)	Critical	CWE-78 / CWE-95	Arbitrary code execution class (RCE)
5	Server-Side Template Injection	High	CWE-79	Template injection / RCE class
6	Path Manipulation / Arbitrary File Read	High	CWE-79	CVE-2024-49766 (Werkzeug safe_join path issue)
7	Insecure Cookie Attributes	Medium	CWE-614	Cookie interception / CSRF class
8	Hardcoded Secret Key	Medium	CWE-259	Hardcoded credential class
9	Container Runs as Root	Medium	CWE-250	Excessive privilege / container escape class